

Multithreading

July 23: I/O-Bound AI Pipelines

Threads • queues • locks • data loaders • throughput



Big idea

Threads help when your program waits on external resources.



Student output

Build and benchmark a threaded dataset loader.

Dong Li
UC Merced

Today's Roadmap

From thread basics to a threaded AI data pipeline

01

Recap

Day 1
bottlenecks
and baselines

02

Threads

lifecycle, GIL,
I/O-bound
logic

03

Thread pools

Futures, map,
submit,
exceptions

04

Coordination

queues, locks,
events,
backpressure

05

Lab

threaded
dataset
loader
benchmark

Learning Objectives

1

Explain

when threading helps and when it does not

2

Use

`threading.Thread` and `ThreadPoolExecutor`

3

Coordinate

work with `Queue`, `Lock`, `Event`, and sentinels

4

Design

producer–consumer data pipelines

5

Measure

throughput, latency, utilization, and speedup

6

Build

a threaded dataset loader for AI preprocessing

1. Recap and Motivation

Day 1 Recap

Parallelism only helps when overhead is smaller than useful work

CPU

CPU-bound

Pure Python computation often needs processes or vectorized libraries, not threads.

I/O

I/O-bound

If tasks wait on files, network, or database, threads can overlap waiting time.

✓

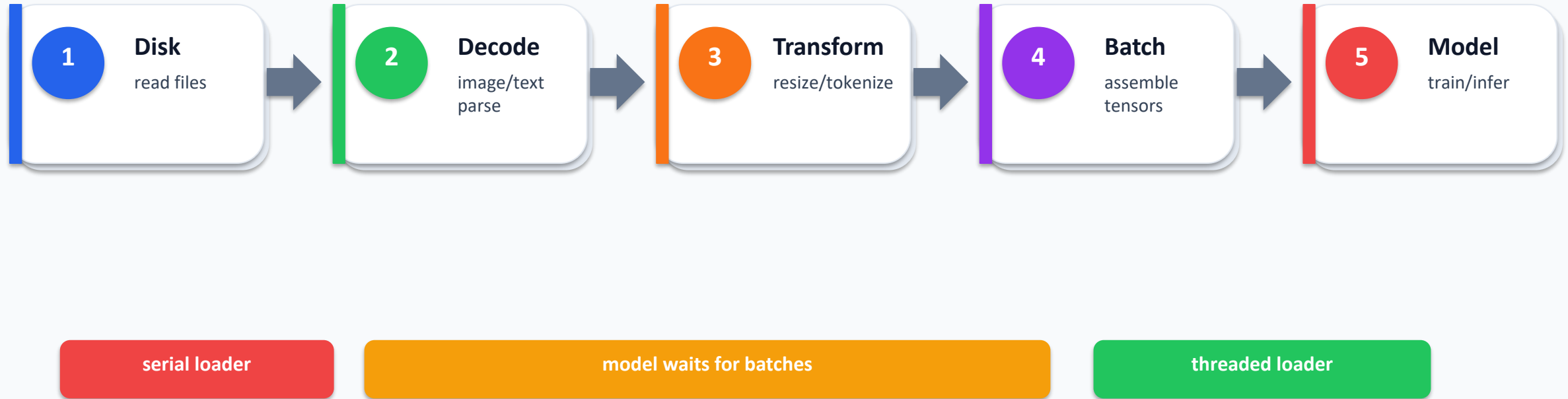
Measure

Always compare to a serial baseline and verify correctness.

Day 2 question: how do we use threads safely and systematically?

Motivation: The Data Loader Bottleneck

A model can be fast but still starve while waiting for data



Where Threads Appear in AI Systems

Threads are often used around the model, not inside the math kernel

DL

Dataset loading

read many files while the model trains

API

Remote calls

fetch data, call APIs, access object storage

LOG

Logging

write metrics without blocking training

REQ

Serving

handle many independent requests

BUF

Pre-fetching

prepare next batch while current batch runs

UI

UI/demo apps

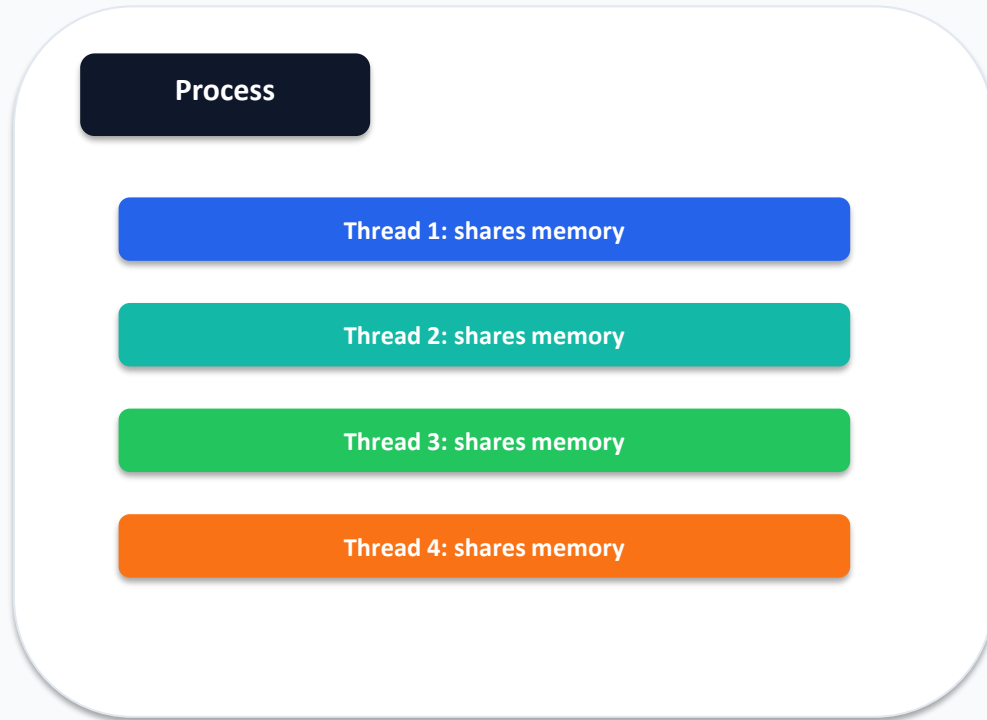
keep interface responsive while model runs

2. Threading Fundamentals

Threads, GIL, lifecycle, and shared memory

What Is a Thread?

A lightweight execution path inside a process



Shared memory

Threads can access the same Python objects.



Coordination needed

Shared mutable data can create race conditions.

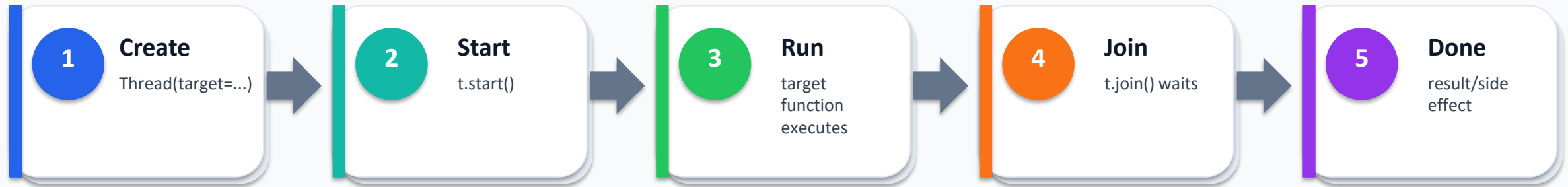


Useful for waiting

Threads can overlap file/network waiting.

Thread Lifecycle

Create → start → run → join



Key mistake: calling the function instead of passing it as target

GIL Reminder: Threads Are Not Magic

Threading is strongest for waiting, not pure Python CPU loops



CPU

CPU-bound Python

Threads usually do not improve pure Python loops.

I/O

I/O-bound waiting

Threads can overlap blocked operations.

C

Native libraries

Some C/NumPy/PyTorch operations may release the GIL.

P

Processes later

Day 3 uses multiprocessing for CPU work.

Example: Manual Threads

Good for understanding; thread pools are usually cleaner



```
import threading
import time

def load_file(path, delay=0.2):
    print(f"start {path}")
    time.sleep(delay)    # simulate file/network waiting
    print(f"done {path}")

paths = [f"file_{i}.jpg" for i in range(5)]
threads = []

for path in paths:
    t = threading.Thread(target=load_file, args=(path,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

print("all files loaded")
```

3. Thread Pools and Futures

Cleaner patterns for many tasks

Why Use a Thread Pool?

Reuse workers instead of manually managing many threads

T

Manual threads

You create, start, track, join, and handle errors yourself.

POOL

Thread pool

A fixed group of workers repeatedly pulls tasks from a work queue.

F

Futures

Objects representing work that may finish later, with results or exceptions.

Use `ThreadPoolExecutor` for most beginner-to-intermediate threaded pipelines

ThreadPoolExecutor: map vs submit

Two common styles for launching threaded work

map

executor.map

Simple when inputs are known and outputs should preserve input order.

Best for: apply one function to many independent items.

sub

submit + as_completed

More flexible. Process results as soon as each task finishes; handle exceptions per task.

Best for: variable task times or robust pipelines.

Example: ThreadPoolExecutor.map

Order-preserving parallel I/O-style work



```
from concurrent.futures import ThreadPoolExecutor
import time

def load_one(path):
    time.sleep(0.1)          # simulate I/O wait
    return {"path": path, "status": "ok"}

paths = [f"image_{i}.jpg" for i in range(20)]

with ThreadPoolExecutor(max_workers=4) as executor:
    results = list(executor.map(load_one, paths))

print(results[:3])
```

Example: submit + as_completed

Process results immediately and handle exceptions

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time, random

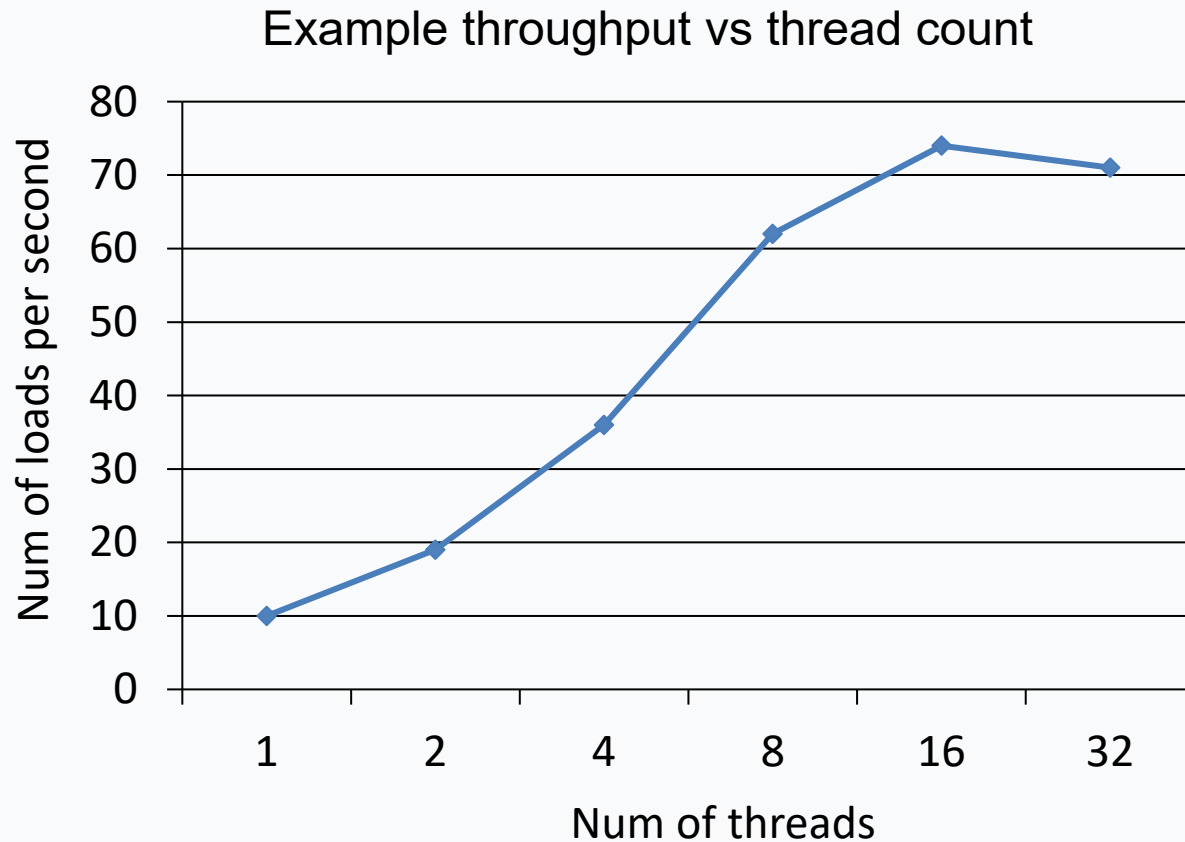
def load_one(path):
    time.sleep(random.uniform(0.05, 0.3))
    if random.random() < 0.05:
        raise ValueError(f"corrupted file: {path}")
    return path, "ok"

paths = [f"image_{i}.jpg" for i in range(50)]

with ThreadPoolExecutor(max_workers=8) as executor:
    futures = {executor.submit(load_one, p): p for p in paths}
    for future in as_completed(futures):
        path = futures[future]
        try:
            result = future.result()
            print("loaded", result)
        except Exception as e:
            print("failed", path, e)
```

Choosing max_workers

More threads can help — until they don't



Too few

workers spend time waiting



Sweet spot

waiting is overlapped effectively



Too many

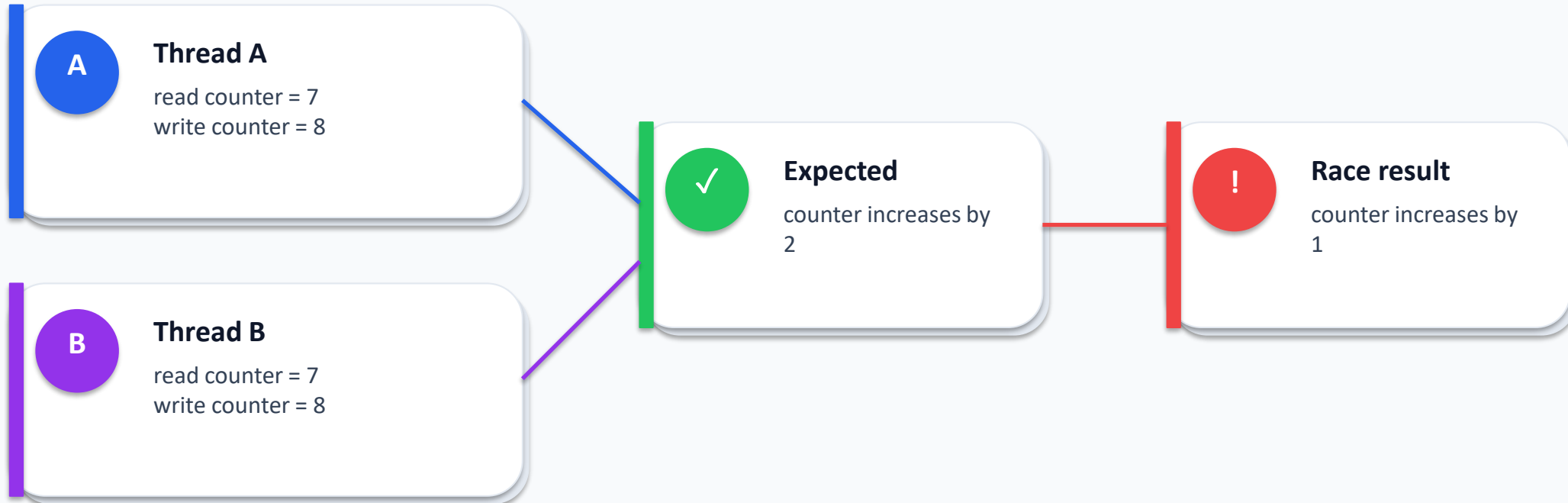
overhead, contention, memory pressure

4. Coordination and Safety

Queues, locks, events, and backpressure

Shared State Risk: Race Conditions

Threads share memory, so correctness matters



Fixes: avoid shared mutable state, use locks or queues, aggregate after workers finish

Lock: Protect a Critical Section

Only one thread enters a protected update at a time



```
import threading

counter = 0
lock = threading.Lock()

def safe_increment(n):
    global counter
    for _ in range(n):
        with lock:            # critical section
            counter += 1

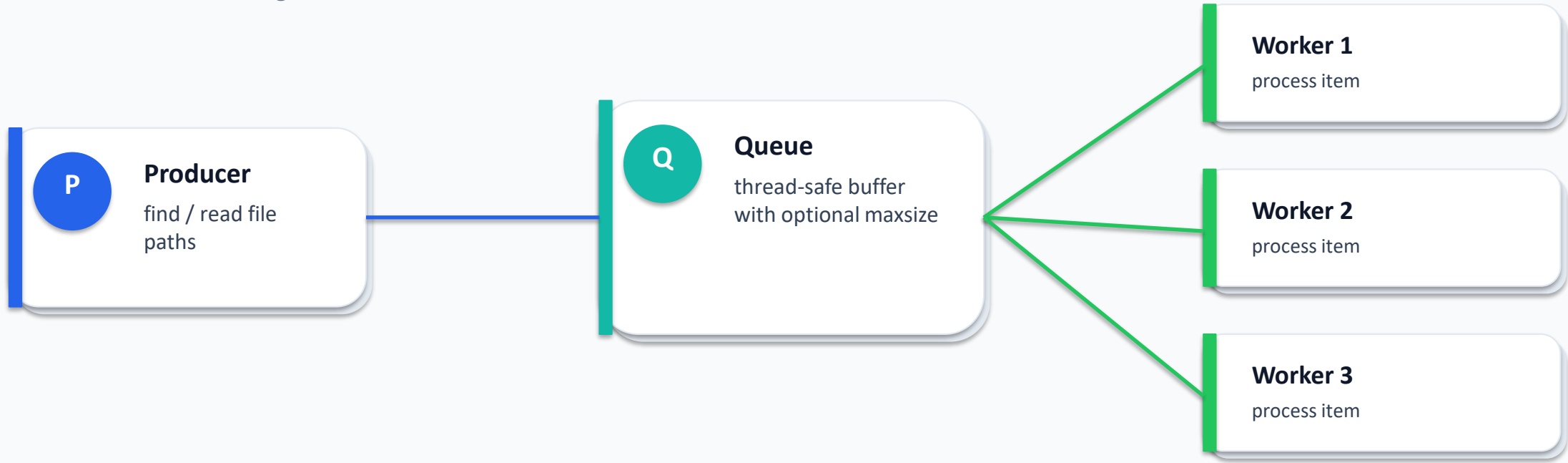
threads = [threading.Thread(target=safe_increment, args=(100_000,))
           for _ in range(4)]

for t in threads: t.start()
for t in threads: t.join()

print(counter)
```

Queue: Producer–Consumer Pattern

A safe buffer between stages



Queue decouples stages and avoids manual locking for each item transfer

Example: Queue with Worker Threads

Use sentinels to tell workers to stop

```
import queue, threading, time

q = queue.Queue(maxsize=20)
results = []
results_lock = threading.Lock()

def worker(worker_id):
    while True:
        item = q.get()
        if item is None:      # sentinel: shutdown signal
            q.task_done()
            break
        time.sleep(0.05)     # simulate loading / preprocessing
        with results_lock:
            results.append((worker_id, item))
        q.task_done()

threads = [threading.Thread(target=worker, args=(i,)) for i in range(4)]
for t in threads: t.start()

for item in range(100):
    q.put(item)

for _ in threads:
    q.put(None)

q.join()
for t in threads: t.join()
print(len(results))
```

Backpressure: Why Bounded Queues Matter

Prevent producers from overwhelming consumers



Unbounded queue

Producer can add millions of tasks; memory can grow unexpectedly.



Bounded queue

Producer blocks when queue is full; memory use is controlled.



Pipeline meaning

Fast stage slows down when downstream is saturated.

Backpressure is a system design idea: control flow by controlling queue capacity

Event: A Simple Signal Between Threads

Coordinate start/stop without busy waiting



```
import threading, time

stop_event = threading.Event()

def monitor():
    while not stop_event.is_set():
        print("monitoring...")
        time.sleep(0.5)
    print("monitor stopped")

t = threading.Thread(target=monitor)
t.start()

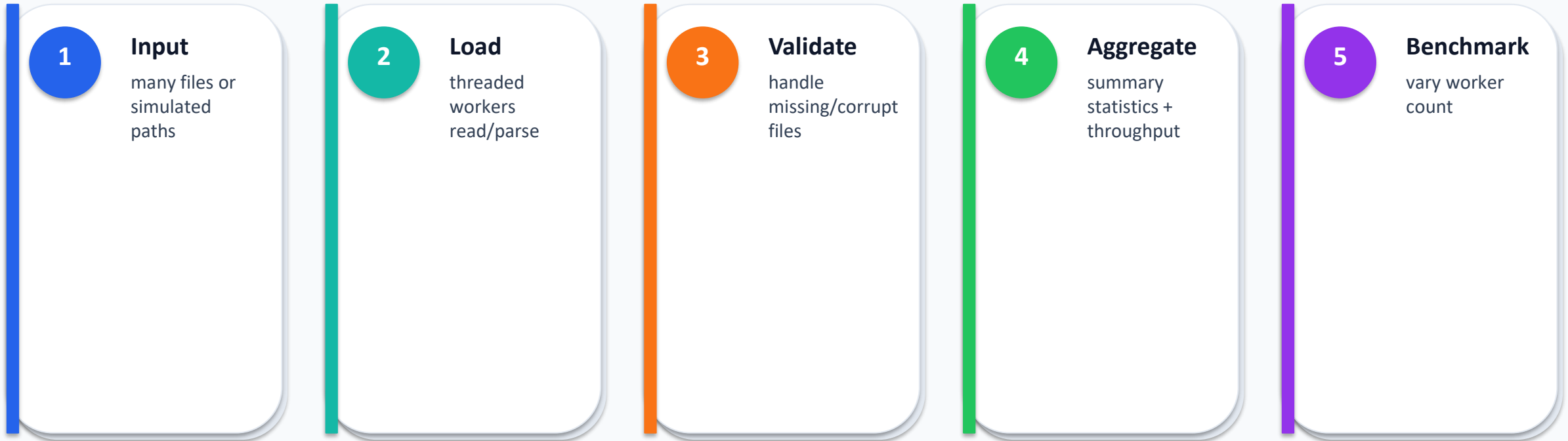
time.sleep(2)
stop_event.set()      # signal worker to stop
t.join()
```

5. Day 2 Lab

Build and benchmark a threaded data loader

Lab: Threaded Dataset Loader

Goal: speed up I/O-bound loading while preserving correctness



Deliverable: results table + short explanation of best worker count

Lab Starter: Simulated File Loader

Works even without an image dataset



```
import time, random

def simulated_load(path):
    # Simulate variable file I/O latency
    time.sleep(random.uniform(0.01, 0.04))

    # Simulate occasional corrupted files
    if random.random() < 0.03:
        raise ValueError(f"corrupted file: {path}")

    # Return fake metadata
    return {
        "path": path,
        "width": random.choice([128, 224, 256, 512]),
        "height": random.choice([128, 224, 256, 512]),
        "brightness": random.random(),
    }
```

Step 1: Serial Baseline

Always measure the simplest correct version first



```
import time

paths = [f"image_{i}.jpg" for i in range(500)]

def load_serial(paths):
    results, errors = [], []
    for path in paths:
        try:
            results.append(simulated_load(path))
        except Exception as e:
            errors.append((path, str(e)))
    return results, errors

start = time.perf_counter()
serial_results, serial_errors = load_serial(paths)
elapsed = time.perf_counter() - start

print("serial time:", elapsed)
print("loaded:", len(serial_results), "errors:", len(serial_errors))
```

Step 2: ThreadPool Loader

Process completed tasks as they finish

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time

def load_threaded(paths, workers=8):
    results, errors = [], []

    with ThreadPoolExecutor(max_workers=workers) as executor:
        futures = {executor.submit(simulated_load, p): p for p in paths}

        for future in as_completed(futures):
            path = futures[future]
            try:
                results.append(future.result())
            except Exception as e:
                errors.append((path, str(e)))

    return results, errors

start = time.perf_counter()
thread_results, thread_errors = load_threaded(paths, workers=8)
elapsed = time.perf_counter() - start

print("threaded time:", elapsed)
print("loaded:", len(thread_results), "errors:", len(thread_errors))
```

Step 3: Benchmark Worker Counts

Find the sweet spot, not the largest number

```
def benchmark_workers(paths, worker_counts=(1, 2, 4, 8, 16, 32)):
    rows = []

    for workers in worker_counts:
        start = time.perf_counter()
        results, errors = load_threaded(paths, workers=workers)
        elapsed = time.perf_counter() - start

        rows.append({
            "workers": workers,
            "time": elapsed,
            "loaded": len(results),
            "errors": len(errors),
            "throughput": len(paths) / elapsed,
        })

    return rows

rows = benchmark_workers(paths)
for row in rows:
    print(row)
```

Step 4: Aggregate Dataset Statistics

Parallel loading must produce useful, reproducible outputs

```
def summarize_metadata(records):  
    if not records:  
        return {}  
  
    avg_width = sum(r["width"] for r in records) / len(records)  
    avg_height = sum(r["height"] for r in records) / len(records)  
    avg_brightness = sum(r["brightness"] for r in records) / len(records)  
  
    return {  
        "count": len(records),  
        "avg_width": avg_width,  
        "avg_height": avg_height,  
        "avg_brightness": avg_brightness,  
    }  
  
summary = summarize_metadata(thread_results)  
print(summary)
```

Optional: Real Image Folder Version

If Pillow and an image folder are available

```
from pathlib import Path
from PIL import Image, ImageStat

IMAGE_DIR = Path("images")
paths = list(IMAGE_DIR.glob("*.jpg")) + list(IMAGE_DIR.glob("*.png"))

def load_image_metadata(path):
    with Image.open(path) as img:
        img = img.convert("RGB")
        width, height = img.size
        stat = ImageStat.Stat(img)
        brightness = sum(stat.mean) / 3.0

    return {
        "path": str(path),
        "width": width,
        "height": height,
        "brightness": brightness,
    }
```

Mini Challenge

Optimize and explain your data loader

1

Run

Benchmark workers = 1, 2, 4, 8, 16, 32.

2

Record

runtime, throughput, errors, and speedup.

3

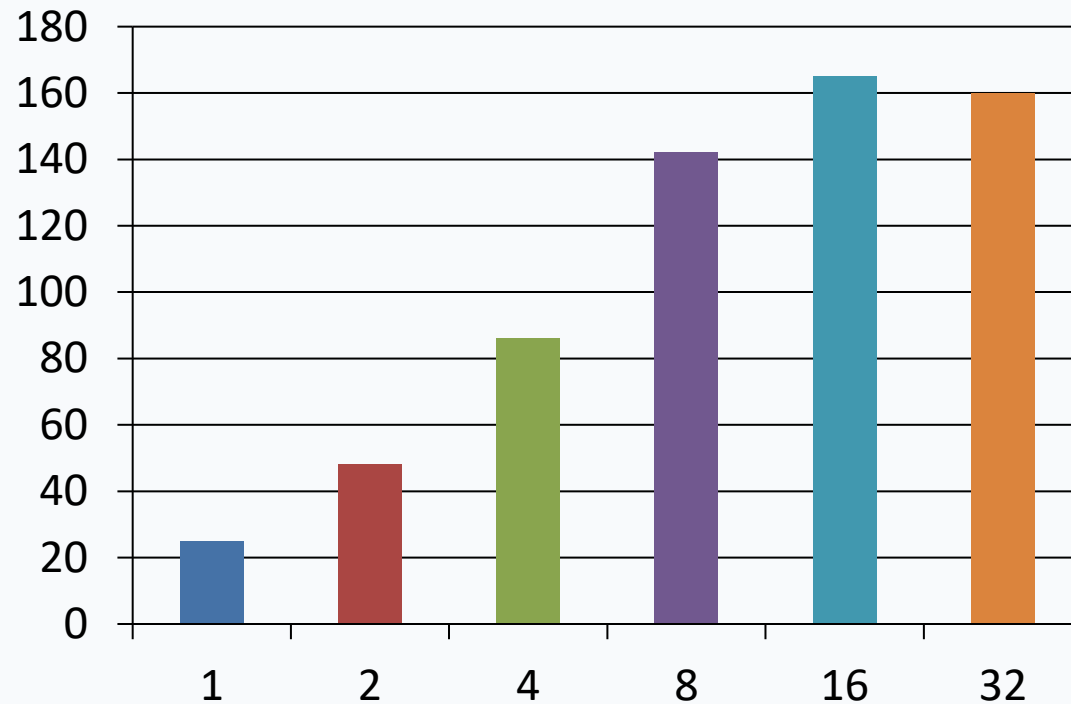
Explain

Why does speedup flatten?
What resource is bottlenecking?

Stretch: add a bounded queue producer–consumer implementation

Expected Result Format

Example throughput vs workers



Best worker count

Highest throughput without unstable errors or memory pressure.



Flattening curve

Disk, scheduling, queueing, or Python overhead may dominate.



Report

Include both speed and correctness.

Exit Ticket

Short reflection + benchmark table

Q

Answer

- Why did threading help or not help?
- What was your best worker count?
- What bottleneck do you suspect?
- How did you handle errors?
- Where could your project use a queue?

✓

Submit

- Code or notebook
- Runtime/throughput table
- Correctness/error counts
- One chart
- 1-paragraph interpretation

References and Further Reading

Threading, futures, and queues

1. **Python threading documentation:** threads are useful for I/O-bound tasks; `ThreadPoolExecutor` is a higher-level interface.
2. **Python `concurrent.futures` documentation:** `Executor`, `Future`, `map`, `submit`, `ThreadPoolExecutor`, `ProcessPoolExecutor`.
3. **Python queue documentation:** `Queue` implements locking semantics for multi-producer, multi-consumer threaded programs.
4. **Optional:** Python logging, `pathlib`, `Pillow` documentation for real image-folder projects.